

Beej's Guide to C Programming

Brian “Beej Jorgensen” Hall

v0.10.5, Copyright © April 18, 2026

Contents

1	Foreword	1
1.1	Audience	1
1.2	How to Read This Book	2
1.3	Platform and Compiler	2
1.4	Official Homepage	2
1.5	Email Policy	2
1.6	Mirroring	3
1.7	Note for Translators	3
1.8	Copyright and Distribution	3
1.9	Dedication	3
2	Hello, World!	5
2.1	What to Expect from C	5
2.2	Hello, World!	6
2.3	Compilation Details	8
2.4	Building with <code>gcc</code>	8
2.5	Building with <code>clang</code>	8
2.6	Building from IDEs	9
2.7	C Versions	9
3	Variables and Statements	11
3.1	Variables	11
3.1.1	Variable Names	11
3.1.2	Variable Types	12
3.1.3	Boolean Types	13
3.2	Operators and Expressions	14
3.2.1	Arithmetic	14
3.2.2	Ternary Operator	15
3.2.3	Pre-and-Post Increment-and-Decrement	15
3.2.4	The Comma Operator	16
3.2.5	Conditional Operators	17
3.2.6	Boolean Operators	17
3.2.7	The <code>sizeof</code> Operator	17
3.3	Flow Control	18
3.3.1	The <code>if - else</code> statement	19
3.3.2	The <code>while</code> statement	20
3.3.3	The <code>do-while</code> statement	21
3.3.4	The <code>for</code> statement	22
3.3.5	The <code>switch</code> Statement	23
4	Functions	26
4.1	Passing by Value	27
4.2	Function Prototypes	28
4.3	Empty Parameter Lists	29
5	Pointers—Cower In Fear!	30
5.1	Memory and Variables	30

5.2	Pointer Types	32
5.3	Dereferencing	33
5.4	Passing Pointers as Arguments	34
5.5	The <code>NULL</code> Pointer	35
5.6	A Note on Declaring Pointers	36
5.7	<code>sizeof</code> and Pointers	36
6	Arrays	38
6.1	Easy Example	38
6.2	Getting the Length of an Array	39
6.3	Array Initializers	39
6.4	Out of Bounds!	41
6.5	Multidimensional Arrays	42
6.6	Arrays and Pointers	43
6.6.1	Getting a Pointer to an Array	43
6.6.2	Passing Single Dimensional Arrays to Functions	43
6.6.3	Changing Arrays in Functions	44
6.6.4	Passing Multidimensional Arrays to Functions	45
7	Strings	47
7.1	String Literals	47
7.2	String Variables	47
7.3	String Variables as Arrays	48
7.4	String Initializers	48
7.5	Getting String Length	49
7.6	String Termination	50
7.7	Copying a String	50
8	Structs	52
8.1	Declaring a Struct	52
8.2	Struct Initializers	53
8.3	Passing Structs to Functions	53
8.4	The Arrow Operator	55
8.5	Copying and Returning <code>struct s</code>	55
8.6	Comparing <code>struct s</code>	55
9	File Input/Output	56
9.1	The <code>FILE*</code> Data Type	56
9.2	Reading Text Files	57
9.3	End of File: <code>EOF</code>	58
9.3.1	Reading a Line at a Time	58
9.4	Formatted Input	59
9.5	Writing Text Files	60
9.6	Binary File I/O	61
9.6.1	<code>struct</code> and Number Caveats	62
10	<code>typedef</code> : Making New Types	64
10.1	<code>typedef</code> in Theory	64
10.1.1	Scoping	64
10.2	<code>typedef</code> in Practice	64
10.2.1	<code>typedef</code> and <code>struct s</code>	64
10.2.2	<code>typedef</code> and Other Types	66
10.2.3	<code>typedef</code> and Pointers	66
10.2.4	<code>typedef</code> and Capitalization	66
10.3	Arrays and <code>typedef</code>	67
11	Pointers II: Arithmetic	68
11.1	Pointer Arithmetic	68

11.1.1	Adding to Pointers	68
11.1.2	Changing Pointers	69
11.1.3	Subtracting Pointers	70
11.2	Array/Pointer Equivalence	71
11.2.1	Array/Pointer Equivalence in Function Calls	72
11.3	<code>void</code> Pointers	72
12	Manual Memory Allocation	77
12.1	Allocating and Deallocating, <code>malloc()</code> and <code>free()</code>	77
12.2	Error Checking	78
12.3	Allocating Space for an Array	79
12.4	An Alternative: <code>calloc()</code>	79
12.5	Changing Allocated Size with <code>realloc()</code>	80
12.5.1	Reading in Lines of Arbitrary Length	81
12.5.2	<code>realloc()</code> with <code>NULL</code>	83
12.6	Aligned Allocations	83
13	Scope	85
13.1	Block Scope	85
13.1.1	Where To Define Variables	85
13.1.2	Variable Hiding	86
13.2	File Scope	86
13.3	<code>for</code> -loop Scope	87
13.4	A Note on Function Scope	87
14	Types II: Way More Types!	88
14.1	Signed and Unsigned Integers	88
14.2	Character Types	89
14.3	More Integer Types: <code>short</code> , <code>long</code> , <code>long long</code>	90
14.4	More Float: <code>double</code> and <code>long double</code>	92
14.4.1	How Many Decimal Digits?	93
14.4.2	Converting to Decimal and Back	94
14.5	Constant Numeric Types	95
14.5.1	Hexadecimal and Octal	95
14.5.2	Integer Constants	96
14.5.3	Floating Point Constants	98
15	Types III: Conversions	100
15.1	String Conversions	100
15.1.1	Numeric Value to String	100
15.1.2	String to Numeric Value	101
15.2	<code>char</code> Conversions	103
15.3	Numeric Conversions	104
15.3.1	Boolean	104
15.3.2	Integer to Integer Conversions	104
15.3.3	Integer and Floating Point Conversions	105
15.4	Implicit Conversions	105
15.4.1	The Integer Promotions	105
15.4.2	The Usual Arithmetic Conversions	105
15.4.3	<code>void*</code>	106
15.5	Explicit Conversions	106
15.5.1	Casting	106
16	Types IV: Qualifiers and Specifiers	108
16.1	Type Qualifiers	108
16.1.1	<code>const</code>	108
16.1.2	<code>restrict</code>	110
16.1.3	<code>volatile</code>	111

16.1.4	<code>_Atomic</code>	111
16.2	Storage-Class Specifiers	112
16.2.1	<code>auto</code>	112
16.2.2	<code>static</code>	112
16.2.3	<code>extern</code>	113
16.2.4	<code>register</code>	114
16.2.5	<code>_Thread_local</code>	115
17	Multifile Projects	116
17.1	Includes and Function Prototypes	116
17.2	Dealing with Repeated Includes	118
17.3	<code>static</code> and <code>extern</code>	119
17.4	Compiling with Object Files	119
18	The Outside Environment	121
18.1	Command Line Arguments	121
18.1.1	The Last <code>argv</code> is <code>NULL</code>	123
18.1.2	The Alternate: <code>char **argv</code>	123
18.1.3	Fun Facts	124
18.2	Exit Status	125
18.2.1	Other Exit Status Values	126
18.3	Environment Variables	127
18.3.1	Setting Environment Variables	128
18.3.2	Unix-like Alternative Environment Variables	128
19	The C Preprocessor	130
19.1	<code>#include</code>	130
19.2	Simple Macros	131
19.3	Conditional Compilation	131
19.3.1	If Defined, <code>#ifdef</code> and <code>#endif</code>	132
19.3.2	If Not Defined, <code>#ifndef</code>	132
19.3.3	<code>#else</code>	133
19.3.4	Else-If: <code>#elifdef</code> , <code>#elifndef</code>	133
19.3.5	General Conditional: <code>#if</code> , <code>#elif</code>	133
19.3.6	Losing a Macro: <code>#undef</code>	135
19.4	Built-in Macros	135
19.4.1	Mandatory Macros	136
19.4.2	Optional Macros	137
19.5	Macros with Arguments	137
19.5.1	Macros with One Argument	137
19.5.2	Macros with More than One Argument	138
19.5.3	Macros with Variable Arguments	139
19.5.4	Stringification	140
19.5.5	Concatenation	140
19.6	Multiline Macros	141
19.7	Example: An Assert Macro	142
19.8	The <code>#error</code> Directive	143
19.9	The <code>#embed</code> Directive	144
19.9.1	<code>#embed</code> Parameters	145
19.9.2	The <code>limit()</code> Parameter	145
19.9.3	The <code>if_empty</code> Parameter	145
19.9.4	The <code>prefix()</code> and <code>suffix()</code> Parameters	146
19.9.5	The <code>__has_embed()</code> Identifier	146
19.9.6	Other Parameters	147
19.9.7	Embedding Multi-Byte Values	148
19.10	The <code>#pragma</code> Directive	148
19.10.1	Non-Standard Pragmas	148

19.10.2	Standard Pragmas	148
19.10.3	<code>_Pragma</code> Operator	149
19.11	The <code>#line</code> Directive	149
19.12	The Null Directive	149
20	<code>struct</code> s II: More Fun with <code>struct</code> s	151
20.1	Initializers of Nested <code>struct</code> s and Arrays	151
20.2	Anonymous <code>struct</code> s	153
20.3	Self-Referential <code>struct</code> s	154
20.4	Flexible Array Members	154
20.5	Padding Bytes	156
20.6	<code>offsetof</code>	156
20.7	Fake OOP	157
20.8	Bit-Fields	159
20.8.1	Non-Adjacent Bit-Fields	159
20.8.2	Signed or Unsigned <code>int</code> s	160
20.8.3	Unnamed Bit-Fields	160
20.8.4	Zero-Width Unnamed Bit-Fields	160
20.9	Unions	161
20.9.1	Unions and Type Punning	161
20.9.2	Pointers to <code>union</code> s	162
20.9.3	Common Initial Sequences in Unions	163
20.10	Unions and Unnamed Structs	165
20.11	Passing and Returning <code>struct</code> s and <code>union</code> s	166
21	Characters and Strings II	168
21.1	Escape Sequences	168
21.1.1	Frequently-used Escapes	168
21.1.2	Rarely-used Escapes	169
21.1.3	Numeric Escapes	171
22	Enumerated Types: <code>enum</code>	172
22.1	Behavior of <code>enum</code>	172
22.1.1	Numbering	172
22.1.2	Trailing Commas	173
22.1.3	Scope	173
22.1.4	Style	173
22.2	Your <code>enum</code> is a Type	173
23	Pointers III: Pointers to Pointers and More	176
23.1	Pointers to Pointers	176
23.1.1	Pointer Pointers and <code>const</code>	178
23.2	Multibyte Values	179
23.3	The <code>NULL</code> Pointer and Zero	181
23.4	Pointers as Integers	181
23.5	Casting Pointers to other Pointers	182
23.6	Pointer Differences	184
23.7	Pointers to Functions	184
24	Bitwise Operations	187
24.1	Bitwise AND, OR, XOR, and NOT	187
24.2	Bitwise Shift	187
25	Variadic Functions	189
25.1	Ellipses in Function Signatures	189
25.2	Getting the Extra Arguments	190
25.3	<code>va_list</code> Functionality	191
25.4	Library Functions That Use <code>va_list</code> s	192
25.5	Variadic Macro Gotchas	192

26	Locale and Internationalization	194
26.1	Setting the Localization, Quick and Dirty	194
26.2	Getting the Monetary Locale Settings	195
26.2.1	Monetary Digit Grouping	196
26.2.2	Separators and Sign Position	197
26.2.3	Example Values	197
26.3	Localization Specifics	198
27	Unicode, Wide Characters, and All That	199
27.1	What is Unicode?	199
27.2	Code Points	199
27.3	Encoding	200
27.4	Source and Execution Character Sets	201
27.5	Unicode in C	201
27.6	A Quick Note on UTF-8 Before We Swerve into the Weeds	202
27.7	Different Character Types	203
27.7.1	Multibyte Characters	203
27.7.2	Wide Characters	204
27.8	Using Wide Characters and <code>wchar_t</code>	204
27.8.1	Multibyte to <code>wchar_t</code> Conversions	205
27.9	Wide Character Functionality	206
27.9.1	<code>wint_t</code>	206
27.9.2	I/O Stream Orientation	206
27.9.3	I/O Functions	207
27.9.4	Type Conversion Functions	207
27.9.5	String and Memory Copying Functions	207
27.9.6	String and Memory Comparing Functions	208
27.9.7	String Searching Functions	208
27.9.8	Length/Miscellaneous Functions	208
27.9.9	Character Classification Functions	208
27.10	Parse State, Restartable Functions	209
27.11	Unicode Encodings and C	211
27.11.1	UTF-8	211
27.11.2	UTF-16, UTF-32, <code>char16_t</code> , and <code>char32_t</code>	211
27.11.3	Multibyte Conversions	212
27.11.4	Third-Party Libraries	213
28	Exiting a Program	214
28.1	Normal Exits	214
28.1.1	Returning From <code>main()</code>	214
28.1.2	<code>exit()</code>	214
28.1.3	Setting Up Exit Handlers with <code>atexit()</code>	215
28.2	Quicker Exits with <code>quick_exit()</code>	215
28.3	Nuke it from Orbit: <code>_Exit()</code>	216
28.4	Exiting Sometimes: <code>assert()</code>	216
28.5	Abnormal Exit: <code>abort()</code>	217
29	Signal Handling	218
29.1	What Are Signals?	218
29.2	Handling Signals with <code>signal()</code>	218
29.3	Writing Signal Handlers	219
29.4	What Can We Actually Do?	221
29.5	Friends Don't Let Friends <code>signal()</code>	223
30	Variable-Length Arrays (VLAs)	224
30.1	The Basics	224
30.2	<code>sizeof</code> and VLAs	225

30.3	Multidimensional VLAs	226
30.4	Passing One-Dimensional VLAs to Functions	226
30.5	Passing Multi-Dimensional VLAs to Functions	227
30.5.1	Partial Multidimensional VLAs	228
30.6	Compatibility with Regular Arrays	228
30.7	<code>typedef</code> and VLAs	229
30.8	Jumping Pitfalls	229
30.9	General Issues	229
31	<code>goto</code>	231
31.1	A Simple Example	231
31.2	Labeled <code>continue</code>	232
31.3	Bailing Out	233
31.4	Labeled <code>break</code>	234
31.5	Multi-level Cleanup	234
31.6	Tail Call Optimization	235
31.7	Restarting Interrupted System Calls	236
31.8	<code>goto</code> and Thread Preemption	237
31.9	<code>goto</code> and Variable Scope	237
31.10	<code>goto</code> and Variable-Length Arrays	238
32	Types Part V: Compound Literals and Generic Selections	240
32.1	Compound Literals	240
32.1.1	Passing Unnamed Objects to Functions	241
32.1.2	Unnamed <code>struct</code> s	241
32.1.3	Pointers to Unnamed Objects	242
32.1.4	Unnamed Objects and Scope	242
32.1.5	Silly Unnamed Object Example	243
32.2	Generic Selections	243
33	Arrays Part II	247
33.1	Type Qualifiers for Arrays in Parameter Lists	247
33.2	<code>static</code> for Arrays in Parameter Lists	247
33.3	Equivalent Initializers	248
34	Long Jumps with <code>setjmp</code> , <code>longjmp</code>	251
34.1	Using <code>setjmp</code> and <code>longjmp</code>	251
34.2	Pitfalls	252
34.2.1	The Values of Local Variables	252
34.2.2	How Much State is Saved?	253
34.2.3	You Can't Name Anything <code>setjmp</code>	253
34.2.4	You Can't <code>setjmp()</code> in a Larger Expression	253
34.2.5	When Can't You <code>longjmp()</code> ?	254
34.2.6	You Can't Pass <code>0</code> to <code>longjmp()</code>	254
34.2.7	<code>longjmp()</code> and Variable Length Arrays	254
35	Incomplete Types	255
35.1	Use Case: Self-Referential Structures	255
35.2	Incomplete Type Error Messages	256
35.3	Other Incomplete Types	256
35.4	Use Case: Arrays in Header Files	257
35.5	Completing Incomplete Types	257
36	Complex Numbers	259
36.1	Complex Types	259
36.2	Assigning Complex Numbers	260
36.3	Constructing, Deconstructing, and Printing	260
36.4	Complex Arithmetic and Comparisons	261

36.5	Complex Math	262
36.5.1	Trigonometry Functions	263
36.5.2	Exponential and Logarithmic Functions	263
36.5.3	Power and Absolute Value Functions	263
36.5.4	Manipulation Functions	263
37	Fixed Width Integer Types	264
37.1	The Bit-Sized Types	264
37.2	Maximum Integer Size Type	265
37.3	Using Fixed Size Constants	265
37.4	Limits of Fixed Size Integers	266
37.5	Format Specifiers	266
38	Date and Time Functionality	268
38.1	Quick Terminology and Information	268
38.2	Date Types	269
38.3	Initialization and Conversion Between Types	269
38.3.1	Converting <code>time_t</code> to <code>struct tm</code>	270
38.3.2	Converting <code>struct tm</code> to <code>time_t</code>	270
38.4	Formatted Date Output	271
38.5	More Resolution with <code>timespec_get()</code>	272
38.6	Differences Between Times	273
39	Multithreading	275
39.1	Background	275
39.2	Things You Can Do	276
39.3	Data Races and the Standard Library	276
39.4	Creating and Waiting for Threads	276
39.5	Detaching Threads	280
39.6	Thread Local Data	281
39.6.1	<code>_Thread_local</code> Storage-Class	283
39.6.2	Another Option: Thread-Specific Storage	283
39.7	Mutexes	285
39.7.1	Different Mutex Types	288
39.8	Condition Variables	288
39.8.1	Timed Condition Wait	292
39.8.2	Broadcast: Wake Up All Waiting Threads	292
39.9	Running a Function One Time	293
40	Atomics	294
40.1	Testing for Atomic Support	294
40.2	Atomic Variables	294
40.3	Synchronization	296
40.4	Acquire and Release	298
40.5	Sequential Consistency	299
40.6	Atomic Assignments and Operators	300
40.7	Library Functions that Automatically Synchronize	300
40.8	Atomic Type Specifier, Qualifier	301
40.9	Lock-Free Atomic Variables	302
40.9.1	Signal Handlers and Lock-Free Atomics	303
40.10	Atomic Flags	304
40.11	Atomic <code>struct</code> s and <code>union</code> s	304
40.12	Atomic Pointers	305
40.13	Memory Order	306
40.13.1	Sequential Consistency	306
40.13.2	Acquire	307
40.13.3	Release	307
40.13.4	Consume	307

40.13.5	Acquire/Release	307
40.13.6	Relaxed	307
40.14	Fences	307
40.15	References	308
41	Function Specifiers, Alignment Specifiers/Operators	309
41.1	Function Specifiers	309
41.1.1	<code>inline</code> for Speed—Maybe	309
41.1.2	<code>noreturn</code> and <code>_Noreturn</code>	310
41.2	Alignment Specifiers and Operators	311
41.2.1	<code>alignas</code> and <code>_Alignas</code>	311
41.2.2	<code>alignof</code> and <code>_Alignof</code>	311
41.3	<code>memalignment()</code> Function	312

Chapter 1

Foreword

C is not a big language, and it is not well served by a big book.

—Brian W. Kernighan, Dennis M. Ritchie

No point in wasting words here, folks, let's jump straight into the C code:

```
E((ck?main((z?(stat(M,&t)?P+=a+'{'?0:3:
execv(M,k),a=G,i=P,y=G&255,
sprintf(Q,y/'@'-3?A(*L(V(%d+%d)+%d,0)
```

And they lived happily ever after. The End.

What's this? You say something's still not clear about this whole C programming language thing?

Well, to be quite honest, I'm not even sure what the above code does. It's a snippet from one of the entries in the 2001 International Obfuscated C Code Contest¹, a wonderful competition wherein the entrants attempt to write the most unreadable C code possible, with often surprising results.

The bad news is that if you're a beginner in this whole thing, all C code you see probably looks obfuscated! The good news is, it's not going to be that way for long.

What we'll try to do over the course of this guide is lead you from complete and utter sheer lost confusion on to the sort of enlightened bliss that can only be obtained through pure C programming. Right on.

In the old days, C was a simpler language. A good number of the features contained in this book and a *lot* of the features in the Library Reference volume didn't exist when K&R wrote the famous second edition of their book in 1988. Nevertheless, the language remains small at its core, and I hope I've presented it here in a way that starts with that simple core and builds outward.

And that's my excuse for writing such a hilariously large book for such a small, concise language.

1.1 Audience

This guide assumes that you've already got some programming knowledge under your belt from another language, such as Python², JavaScript³, Java⁴, Rust⁵, Go⁶, Swift⁷, etc. (Objective-C⁸ devs will have a particularly easy time of it!)

We're going to assume you know what variables are, what loops do, how functions work, and so on.

¹<https://www.ioccc.org/>

²[https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language))

³<https://en.wikipedia.org/wiki/JavaScript>

⁴[https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language))

⁵[https://en.wikipedia.org/wiki/Rust_\(programming_language\)](https://en.wikipedia.org/wiki/Rust_(programming_language))

⁶[https://en.wikipedia.org/wiki/Go_\(programming_language\)](https://en.wikipedia.org/wiki/Go_(programming_language))

⁷[https://en.wikipedia.org/wiki/Swift_\(programming_language\)](https://en.wikipedia.org/wiki/Swift_(programming_language))

⁸<https://en.wikipedia.org/wiki/Objective-C>

If that's not you for whatever reason the best I can hope to provide is some honest entertainment for your reading pleasure. The only thing I can reasonably promise is that this guide won't end on a cliffhanger... or *will* it?

1.2 How to Read This Book

The guide is in two volumes, and this is the first: the tutorial volume!

The second volume is the library reference⁹, and it's far more reference than tutorial.

If you're new, go through the tutorial part in order, generally. The higher you get in chapters, the less important it is to go in order.

And no matter your skill level, the reference part is there with complete examples of the standard library function calls to help refresh your memory whenever needed. Good for reading over a bowl of cereal or other time.

Finally, glancing at the index (if you're reading the print version), the reference section entries are italicized.

1.3 Platform and Compiler

I'll try to stick to Plain Ol'-Fashioned ISO-standard C¹⁰. Well, for the most part. Here and there I might go crazy and start talking about POSIX¹¹ or something, but we'll see.

Unix users (e.g. Linux, BSD, etc.) try running `cc` or `gcc` from the command line—you might already have a compiler installed. If you don't, search your distribution for installing `gcc` or `clang`.

Windows users should check out Visual Studio Community¹². Or, if you're looking for a more Unix-like experience (recommended!), install WSL¹³ and `gcc`.

Mac users will want to install XCode¹⁴, and in particular the command line tools.

There are a lot of compilers out there, and virtually all of them will work for this book. And a C++ compiler will compile a lot of (but not all!) C code. Best use a proper C compiler if you can.

1.4 Official Homepage

This official location of this document is <https://beej.us/guide/bgc/>¹⁵. Maybe this'll change in the future, but it's more likely that all the other guides are migrated off Chico State computers.

1.5 Email Policy

I'm generally available to help out with email questions so feel free to write in, but I can't guarantee a response. I lead a pretty busy life and there are times when I just can't answer a question you have. When that's the case, I usually just delete the message. It's nothing personal; I just won't ever have the time to give the detailed answer you require.

As a rule, the more complex the question, the less likely I am to respond. If you can narrow down your question before mailing it and be sure to include any pertinent information (like platform, compiler, error messages you're getting, and anything else you think might help me troubleshoot), you're much more likely to get a response.

⁹<https://beej.us/guide/bgclr/>

¹⁰https://en.wikipedia.org/wiki/ANSI_C

¹¹<https://en.wikipedia.org/wiki/POSIX>

¹²<https://visualstudio.microsoft.com/vs/community/>

¹³<https://docs.microsoft.com/en-us/windows/wsl/install-win10>

¹⁴<https://developer.apple.com/xcode/>

¹⁵<https://beej.us/guide/bgc/>

If you don't get a response, hack on it some more, try to find the answer, and if it's still elusive, then write me again with the information you've found and hopefully it will be enough for me to help out.

Now that I've badgered you about how to write and not write me, I'd just like to let you know that I *fully* appreciate all the praise the guide has received over the years. It's a real morale boost, and it gladdens me to hear that it is being used for good! :-) Thank you!

1.6 Mirroring

You are more than welcome to mirror this site, whether publicly or privately. If you publicly mirror the site and want me to link to it from the main page, drop me a line at `beej@beej.us`.

1.7 Note for Translators

If you want to translate the guide into another language, write me at `beej@beej.us` and I'll link to your translation from the main page. Feel free to add your name and contact info to the translation.

Please note the license restrictions in the Copyright and Distribution section, below.

1.8 Copyright and Distribution

Beej's Guide to C is Copyright © 2021 Brian "Beej Jorgensen" Hall.

With specific exceptions for source code and translations, below, this work is licensed under the Creative Commons Attribution-Noncommercial-No Derivative Works 3.0 License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-nd/3.0/> or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

One specific exception to the "No Derivative Works" portion of the license is as follows: this guide may be freely translated into any language, provided the translation is accurate, and the guide is reprinted in its entirety. The same license restrictions apply to the translation as to the original guide. The translation may also include the name and contact information for the translator.

The C source code presented in this document is hereby granted to the public domain, and is completely free of any license restriction.

Educators are freely encouraged to recommend or supply copies of this guide to their students.

Contact `beej@beej.us` for more information.

1.9 Dedication

The hardest things about writing these guides are:

- Learning the material in enough detail to be able to explain it
- Figuring out the best way to explain it clearly, a seemingly-endless iterative process
- Putting myself out there as a so-called *authority*, when really I'm just a regular human trying to make sense of it all, just like everyone else
- Keeping at it when so many other things draw my attention

A lot of people have helped me through this process, and I want to acknowledge those who have made this book possible.

- Everyone on the Internet who decided to help share their knowledge in one form or another. The free sharing of instructive information is what makes the Internet the great place that it is.
- The volunteers at [cppreference.com](https://en.cppreference.com/)¹⁶ who provide the bridge that leads from the spec to the real world.

¹⁶<https://en.cppreference.com/>

- The helpful and knowledgeable folks on `comp.lang.c`¹⁷ and `r/C_Programming`¹⁸ who got me through the tougher parts of the language.
- Everyone who submitted corrections and pull-requests on everything from misleading instructions to typos.

Thank you! ♥

¹⁷<https://groups.google.com/g/comp.lang.c>

¹⁸https://www.reddit.com/r/C_Programming/

Chapter 2

Hello, World!

2.1 What to Expect from C

“Where do these stairs go?”

“They go up.”

—Ray Stantz and Peter Venkman, Ghostbusters

C is a low-level language.

It didn’t use to be. Back in the day when people carved punch cards out of granite, C was an incredible way to be free of the drudgery of lower-level languages like assembly¹.

But now in these modern times, current-generation languages offer all kinds of features that didn’t exist in 1972 when C was invented. This means C is a pretty basic language with not a lot of features. It can do *anything*, but it can make you work for it.

So why would we even use it today?

- As a learning tool: not only is C a venerable piece of computing history, but it is connected to the bare metal² in a way that present-day languages are not. When you learn C, you learn about how software interfaces with computer memory at a low level. There are no seatbelts. You’ll write software that crashes, I assure you. And that’s all part of the fun!
- As a useful tool: C still is used for certain applications, such as building operating systems³ or in embedded systems⁴. (Though the Rust⁵ programming language is eyeing both these fields!)

If you’re familiar with another language, a lot of things about C are easy. C inspired many other languages, and you’ll see bits of it in Go, Rust, Swift, Python, JavaScript, Java, and all kinds of other languages. Those parts will be familiar.

The one thing about C that hangs people up is *pointers*. Virtually everything else is familiar, but pointers are the weird one. The concept behind pointers is likely one you already know, but C forces you to be explicit about it, using operators you’ve likely never seen before.

It’s especially insidious because once you *grok*⁶ pointers, they’re suddenly easy. But up until that moment, they’re slippery eels.

Everything else in C is just memorizing another way (or sometimes the same way!) of doing something you’ve done already. Pointers are the weird bit. And, arguably, even pointers are variations on a theme you’re probably familiar with.

¹https://en.wikipedia.org/wiki/Assembly_language

²https://en.wikipedia.org/wiki/Bare_machine

³https://en.wikipedia.org/wiki/Operating_system

⁴https://en.wikipedia.org/wiki/Embedded_system

⁵[https://en.wikipedia.org/wiki/Rust_\(programming_language\)](https://en.wikipedia.org/wiki/Rust_(programming_language))

⁶<https://en.wikipedia.org/wiki/Grok>

So get ready for a rollicking adventure as close to the core of the computer as you can get without assembly, in the most influential computer language of all time⁷. Hang on!

2.2 Hello, World!

This is the canonical example of a C program. Everyone uses it. (Note that the numbers to the left are for reader reference only, and are not part of the source code.)

```

1  /* Hello world program */
2
3  #include <stdio.h>
4
5  int main(void)
6  {
7      printf("Hello, World!\n"); // Actually do the work here
8  }
```

We’re going to don our long-sleeved heavy-duty rubber gloves, grab a scalpel, and rip into this thing to see what makes it tick. So, scrub up, because here we go. Cutting very gently...

Let’s get the easy thing out of the way: anything between the digraphs `/*` and `*/` is a comment and will be completely ignored by the compiler. Same goes for anything on a line after a `//`. This allows you to leave messages to yourself and others, so that when you come back and read your code in the distant future, you’ll know what the heck it was you were trying to do. Believe me, you will forget; it happens.

Now, what is this `#include`? GROSS! Well, it tells the C Preprocessor to pull the contents of another file and insert it into the code right *there*.

Wait—what’s a C Preprocessor? Good question. There are two stages⁸ to compilation: the preprocessor and the compiler. Anything that starts with pound sign, hash symbol, or “octothorpe”, (`#`) is something the preprocessor operates on before the compiler even gets started. Common *preprocessor directives*, as they’re called, are `#include` and `#define`. More on that later.

Before we go on, why would I even begin to bother pointing out that a pound sign is called an octothorpe? The answer is simple: I think the word octothorpe is so excellently funny, I have to gratuitously spread its name around whenever I get the opportunity. Octothorpe. Octothorpe, octothorpe, octothorpe.

So *anyway*. After the C preprocessor has finished preprocessing everything, the results are ready for the compiler to take them and produce assembly code⁹, machine code¹⁰, or whatever it’s about to do. Machine code is the “language” the CPU understands, and it can understand it *very rapidly*. This is one of the reasons C programs tend to be quick.

Don’t worry about the technical details of compilation for now; just know that your source runs through the preprocessor, then the output of that runs through the compiler, then that produces an executable for you to run.

What about the rest of the line? What’s `<stdio.h>`? That is what is known as a *header file*. It’s the dot-h at the end that gives it away. In fact it’s the “Standard I/O” (`stdio`) header file that you will grow to know and love. It gives us access to a bunch of I/O functionality¹¹. For our demo program, we’re outputting the string “Hello, World!”, so we in particular need access to the `printf()` function to do this. The `<stdio.h>` file gives us this access. Basically, if we tried to use `printf()` without `#include <stdio.h>`, the compiler would have complained to us about it.

⁷I know someone will fight me on that, but it’s gotta be at least in the top three, right?

⁸Well, technically there are more than two, but hey, let’s pretend there are two—ignorance is bliss, right?

⁹https://en.wikipedia.org/wiki/Assembly_language

¹⁰https://en.wikipedia.org/wiki/Machine_code

¹¹Technically, it contains preprocessor directives and function prototypes (more on that later) for common input and output needs.

How did I know I needed to `#include <stdio.h>` for `printf()` ? Answer: it's in the documentation. If you're on a Unix system, `man 3 printf` and it'll tell you right at the top of the man page what header files are required. Or see the reference section in this book. :-)

Holy moly. That was all to cover the first line! But, let's face it, it has been completely dissected. No mystery shall remain!

So take a breather...look back over the sample code. Only a couple easy lines to go.

Welcome back from your break! I know you didn't really take a break; I was just humoring you.

The next line is `main()`. This is the definition of the function `main()`; everything between the squirrely braces (`{` and `}`) is part of the function definition.

(How do you *call* a different function, anyway? The answer lies in the `printf()` line, but we'll get to that in a minute.)

Now, the `main` function is a special one in many ways, but one way stands above the rest: it is the function that will be called automatically when your program starts executing. Nothing of yours gets called before `main()`. In the case of our example, this works fine since all we want to do is print a line and exit.

Oh, that's another thing: once the program executes past the end of `main()`, down there at the closing squirrely brace, the program will exit, and you'll be back at your command prompt.

So now we know that that program has brought in a header file, `stdio.h`, and declared a `main()` function that will execute when the program is started. What are the goodies in `main()` ?

I am so happy you asked. Really! We only have the one goodie: a call to the function `printf()`. You can tell this is a function call and not a function definition in a number of ways, but one indicator is the lack of squirrely braces after it. And you end the function call with a semicolon so the compiler knows it's the end of the expression. You'll be putting semicolons after almost everything, as you'll see.

You're passing one argument to the function `printf()`: a string to be printed when you call it. Oh, yeah—we're calling a function! We rock! Wait, wait—don't get cocky. What's that crazy `\n` at the end of the string? Well, most characters in the string will print out just like they are stored. But there are certain characters that you can't print on screen well that are embedded as two-character backslash codes. One of the most popular is `\n` (read "backslash-N" or simply "newline") that corresponds to the *newline* character. This is the character that causes further printing to continue at the beginning of the next line instead of the current. It's like hitting return at the end of the line.

So copy that code into a file called `hello.c` and build it. On a Unix-like platform (e.g. Linux, BSD, Mac, or WSL), from the command line you'll build with a command like so:

```
gcc -o hello hello.c
```

(This means "compile `hello.c`, and output an executable called `hello`".)

After that's done, you should have a file called `hello` that you can run with this command:

```
./hello
```

(The leading `./` tells the shell to "run from the current directory".)

And see what happens:

```
Hello, World!
```

It's done and tested! Ship it!

2.3 Compilation Details

Let’s talk a bit more about how to build C programs, and what happens behind the scenes there.

Like other languages, C has *source code*. But, depending on what language you’re coming from, you might never have had to *compile* your source code into an *executable*.

Compilation is the process of taking your C source code and turning it into a program that your operating system can execute.

JavaScript and Python devs aren’t used to a separate compilation step at all—though behind the scenes it’s happening! Python compiles your source code into something called *bytecode* that the Python virtual machine can execute. Java devs are used to compilation, but that produces bytecode for the Java Virtual Machine.

When compiling C, *machine code* is generated. This is the 1s and 0s that can be executed directly and speedily by the CPU.

Languages that typically aren’t compiled are called *interpreted* languages. But as we mentioned with Java and Python, they also have a compilation step. And there’s no rule saying that C can’t be interpreted. (There are C interpreters out there!) In short, it’s a bunch of gray areas. Compilation in general is just taking source code and turning it into another, more easily-executed form.

The C compiler is the program that does the compilation.

As we’ve already said, `gcc` is a compiler that’s installed on a lot of Unix-like operating systems¹². And it’s commonly run from the command line in a terminal, but not always. You can run it from your IDE, as well.

So how do we do command line builds?

2.4 Building with `gcc`

If you have a source file called `hello.c` in the current directory, you can build that into a program called `hello` with this command typed in a terminal:

```
gcc -o hello hello.c
```

The `-o` means “output to this file”¹³. And there’s `hello.c` at the end, the name of the file we want to compile.

If your source is broken up into multiple files, you can compile them all together (almost as if they were one file, but the rules are actually more complex than that) by putting all the `.c` files on the command line:

```
gcc -o awesomegame ui.c characters.c npc.c items.c
```

and they’ll all get built together into a big executable.

That’s enough to get started—later we’ll talk details about multiple source files, object files, and all kinds of fun stuff.

2.5 Building with `clang`

On Macs, the stock compiler isn’t `gcc`—it’s `clang`. But a wrapper is also installed so you can run `gcc` and have it still work.

¹²<https://en.wikipedia.org/wiki/Unix>

¹³If you don’t give it an output filename, it will export to a file called `a.out` by default—this filename has its roots deep in Unix history.

You can also install the `gcc` compiler proper through Homebrew¹⁴ or some other means.

2.6 Building from IDEs

If you're using an *Integrated Development Environment* (IDE), you probably don't have to build from the command line.

With Visual Studio, `CTRL-F7` will build, and `CTRL-F5` will run.

With VS Code, you can hit `F5` to run via the debugger. (You'll have to install the C/C++ Extension.)

With XCode, you can build with `COMMAND-B` and run with `COMMAND-R`. To get the command line tools, Google for "XCode command line tools" and you'll find instructions for installing them.

For getting started, I encourage you to also try to build from the command line—it's history!

2.7 C Versions

C has come a long way over the years, and it had many named version numbers to describe which dialect of the language you're using.

These generally refer to the year of the specification.

The most famous are C89, C99, C11, and C23. We'll focus on the last one in this book.

But here's a more complete table:

Version	Description
K&R C	1978, the original. Named after Brian Kernighan and Dennis Ritchie. Ritchie designed and coded the language, and Kernighan co-authored the book on it. You rarely see original K&R code today. If you do, it'll look odd, like Middle English looks odd to modern English readers.
C89, ANSI C, C90	In 1989, the American National Standards Institute (ANSI) produced a C language specification that set the tone for C that persists to this day. A year later, the reins were handed to the International Organization for Standardization (ISO) that produced the identical C90.
C95	A rarely-mentioned addition to C89 that included wide character support.
C99	The first big overhaul with lots of language additions. The thing most people remember is the addition of <code>//</code> -style comments. This is the most popular version of C in use as of this writing.
C11	This major version update includes Unicode support and multi-threading. Be advised that if you start using these language features, you might be sacrificing portability with places that are stuck in C99 land. But, honestly, 1999 is getting to be a while back now.
C17, C18	Bugfix update to C11. C17 seems to be the official name, but the publication was delayed until 2018. As far as I can tell, these two are interchangeable, with C17 being preferred.
C23	The most recent specification.

You can force GCC to use one of these standards with the `-std=` command line argument. If you want it to be picky about the standard, add `-pedantic`.

For example:

```
gcc -std=c11 -pedantic foo.c
```

For this book, I compile programs for C23 with all warnings set:

¹⁴<https://formulae.brew.sh/formula/gcc>

```
gcc -Wall -Wextra -std=c23 -pedantic foo.c
```

Chapter 3

Variables and Statements

“It takes all kinds to make a world, does it not, Padre?”

“So it does, my son, so it does.”

—Pirate Captain Thomas Bartholomew Red to the Padre, Pirates

There sure can be lotsa stuff in a C program.

Yup.

And for various reasons, it’ll be easier for all of us if we classify some of the types of things you can find in a program, so we can be clear what we’re talking about.

3.1 Variables

It’s said that “variables hold values”. But another way to think about it is that a variable is a human-readable name that refers to some data in memory.

We’re going to take a second here and take a peek down the rabbit hole that is pointers. Don’t worry about it.

You can think of memory as a big array of bytes¹. Data is stored in this “array”². If a number is larger than a single byte, it is stored in multiple bytes. Because memory is like an array, each byte of memory can be referred to by its index. This index into memory is also called an *address*, or a *location*, or a *pointer*.

When you have a variable in C, the value of that variable is in memory *somewhere*, at some address. Of course. After all, where else would it be? But it’s a pain to refer to a value by its numeric address, so we make a name for it instead, and that’s what the variable is.

The reason I’m bringing all this up is twofold:

1. It’s going to make it easier to understand pointer variables later—they’re variables that hold the address of other variables!
2. Also, it’s going to make it easier to understand pointers later.

So a variable is a name for some data that’s stored in memory at some address.

3.1.1 Variable Names

You can use any characters in the range 0-9, A-Z, a-z, and underscore for variable names, with the following rules:

- You can’t start a variable with a digit 0-9.

¹A “byte” is typically an 8-bit binary number. Think of it as an integer that can only hold the values from 0 to 255, inclusive. Technically, C allows bytes to be any number of bits and if you want to unambiguously refer to an 8-bit number, you should use the term *octet*. But programmers are going to assume you mean 8-bits when you say “byte” unless you specify otherwise.

²I’m seriously oversimplifying how modern memory works, here. But the mental model works, so please forgive me.

- You can't start a variable name with two underscores.
- You can't start a variable name with an underscore followed by a capital A-Z.

For Unicode, just try it. There are some rules in the spec in §D.2 that talk about which Unicode codepoint ranges are allowed in which parts of identifiers, but that's too much to write about here and is probably something you'll never have to think about anyway.

3.1.2 Variable Types

Depending on which languages you already have in your toolkit, you might or might not be familiar with the idea of types. But C's kinda picky about them, so we should do a refresher.

Some example types, some of the most basic:

Type	Example	C Type
Integer	3490	int
Floating point	3.14159	float ³
Character (single)	'c'	char
String	"Hello, world!"	char * ⁴

C makes an effort to convert automatically between most numeric types when you ask it to. But other than that, all conversions are manual, notably between string and numeric.

Almost all of the types in C are variants on these types.

Before you can use a variable, you have to *declare* that variable and tell C what type the variable holds. Once declared, the type of variable cannot be changed later at runtime. What you set it to is what it is until it falls out of scope and is reabsorbed into the universe.

Let's take our previous "Hello, world" code and add a couple variables to it:

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i;    // Holds signed integers, e.g. -3, -2, 0, 1, 10
6      float f;  // Holds signed floating point numbers, e.g. -3.1416
7
8      printf("Hello, World!\n"); // Ah, blessed familiarity
9  }
```

There! We've declared a couple of variables. We haven't used them yet, and they're both uninitialized. One holds an integer number, and the other holds a floating point number (a real number, basically, if you have a math background).

Uninitialized variables have indeterminate value⁵. They have to be initialized or else you must assume they contain some nonsense number.

This is one of the places C can "get you". Much of the time, in my experience, the indeterminate value is zero... but it can vary from run to run! Never assume the value will be zero, even if you see it is. *Always* explicitly initialize variables to some value before you use them⁴.

⁴This isn't strictly 100% true. When we get to learning about static storage duration, you'll find the some variables are initialized to zero automatically. But the safe thing to do is always initialize them.

What's this? You want to store some numbers in those variables? Insanity!

³I'm lying here a little. Technically 3.14159 is of type `double`, but we're not there yet and I want you to associate `float` with "Floating Point", and C will happily coerce that type into a `float`. In short, don't worry about it until later.

⁴Read this as "pointer to a char" or "char pointer". "Char" for character. Though I can't find a study, it seems anecdotally most people pronounce this as "char", a minority say "car", and a handful say "care". We'll talk more about pointers later.

⁵Colloquially, we say they have "random" values, but they aren't truly—or even pseudo-truly—random numbers.

Let's go ahead and do that:

```

1  int main(void)
2  {
3      int i;
4
5      i = 2; // Assign the value 2 into the variable i
6
7      printf("Hello, World!\n");
8  }
```

Killer. We've stored a value. Let's print it.

We're going to do that by passing *two* amazing arguments to the `printf()` function. The first argument is a string that describes what to print and how to print it (called the *format string*), and the second is the value to print, namely whatever is in the variable `i`.

`printf()` hunts through the format string for a variety of special sequences which start with a percent sign (`%`) that tell it what to print. For example, if it finds a `%d`, it looks to the next parameter that was passed, and prints it out as an integer. If it finds a `%f`, it prints the value out as a float. If it finds a `%s`, it prints a string.

As such, we can print out the value of various types like so:

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i = 2;
6      float f = 3.14;
7      char *s = "Hello, world!"; // char * ("char pointer") is the string type
8
9      printf("%s i = %d and f = %f!\n", s, i, f);
10 }
```

And the output will be:

```
Hello, world! i = 2 and f = 3.14!
```

In this way, `printf()` might be similar to various types of format strings or parameterized strings in other languages you're familiar with.

NOTE: In the following sections, I just assume that you've declared the variables ahead of time. If the example uses integers `i` and `j`, just assume somewhere above the example I had:

```
int i, j;
```

3.1.3 Boolean Types

C has Boolean types, true or false?

```
1!
```

Historically, C didn't have a Boolean type, and some might argue it still doesn't.

In C, `0` means "false", and non-zero means "true".

So `1` is true. And `-37` is true. And `0` is false.

You can just declare Boolean types as `int` s:

```
int x = 1;

if (x) {
    printf("x is true!\n");
}
```

In C23, you get actual `bool`, `true`, and `false`. Before that, if you have a modern-enough version of C, you can `#include <stdbool.h>` to get the same thing.

```
1 #include <stdio.h>
2 #include <stdbool.h> // not needed in C23
3
4 int main(void) {
5     bool x = true;
6
7     if (x) {
8         printf("x is true!\n");
9     }
10 }
```

While technically you should be setting a `bool` variable to `true`, `false`, or the result of some expression that evaluates to true or false, you can actually convert all kinds of things to `bool`. There are some specific rules, but zero-ish things tend to evaluate to `false`, and non-zero-ish things to `true`.

But be careful if you mix and match since the numeric value of `true` is `1`, probably⁶, and if you're relying on some other positive value to be true, you might get a mismatch. For example:

```
1 printf("%d\n", true == 12); // Prints "0", false!
```

3.2 Operators and Expressions

C operators should be familiar to you from other languages. Let's blast through some of them here.

(There are a bunch more details than this, but we're going to do enough in this section to get started.)

3.2.1 Arithmetic

Hopefully these are familiar:

```
i = i + 3; // Addition (+) and assignment (=) operators, add 3 to i
i = i - 8; // Subtraction, subtract 8 from i
i = i * 9; // Multiplication
i = i / 2; // Division
i = i % 5; // Modulo (division remainder)
```

There are shorthand variants for all of the above. Each of those lines could more tersely be written as:

```
i += 3; // Same as "i = i + 3", add 3 to i
i -= 8; // Same as "i = i - 8"
i *= 9; // Same as "i = i * 9"
i /= 2; // Same as "i = i / 2"
i %= 5; // Same as "i = i % 5"
```

⁶Technically just one bit of a `char` is used to represent the `bool`, so it can either be zero or one. Except that what goes in the remaining (padding) bits of the `char` is unspecified. For `false`, it must surely be all zero. But for `true`, I'm uncertain that it must all be zero.

There is no exponentiation. You'll have to use one of the `pow()` function variants from `math.h`. Let's get into some of the weirder stuff you might not have in your other languages!

3.2.2 Ternary Operator

C also includes the *ternary operator*. This is an expression whose value depends on the result of a conditional embedded in it.

```
// If x > 10, add 17 to y. Otherwise add 37 to y.
y += x > 10? 17: 37;
```

What a mess! You'll get used to it the more you read it. To help out a bit, I'll rewrite the above expression using `if` statements:

```
// This expression:
y += x > 10? 17: 37;

// is equivalent to this non-expression:
if (x > 10)
    y += 17;
else
    y += 37;
```

Compare those two until you see each of the components of the ternary operator.

Or, another example that prints if a number stored in `x` is odd or even:

```
printf("The number %d is %s.\n", x, x % 2 == 0? "even": "odd");
```

The `%s` format specifier in `printf()` means print a string. If the expression `x % 2` evaluates to `0`, the value of the entire ternary expression evaluates to the string `"even"`. Otherwise it evaluates to the string `"odd"`. Pretty cool!

It's important to note that the ternary operator isn't flow control like the `if` statement is. It's just an expression that evaluates to a value.

3.2.3 Pre-and-Post Increment-and-Decrement

Now, let's mess with another thing that you might not have seen.

These are the legendary post-increment and post-decrement operators:

```
i++;      // Add one to i (post-increment)
i--;      // Subtract one from i (post-decrement)
```

Very commonly, these are just used as shorter versions of:

```
i += 1;    // Add one to i
i -= 1;    // Subtract one from i
```

but they're more subtly different than that, the clever scoundrels.

Let's take a look at this variant, pre-increment and pre-decrement:

```
++i;      // Add one to i (pre-increment)
--i;      // Subtract one from i (pre-decrement)
```

With pre-increment and pre-decrement, the value of the variable is incremented or decremented *before* the expression is evaluated. Then the expression is evaluated with the new value.

With post-increment and post-decrement, the value of the expression is first computed with the value as-is, and *then* the value is incremented or decremented after the value of the expression has been determined.

You can actually embed them in expressions, like this:

```
i = 10;
j = 5 + i++; // Compute 5 + i, _then_ increment i

printf("%d, %d\n", i, j); // Prints 11, 15
```

Let's compare this to the pre-increment operator:

```
i = 10;
j = 5 + ++i; // Increment i, _then_ compute 5 + i

printf("%d, %d\n", i, j); // Prints 11, 16
```

This technique is used frequently with array and pointer access and manipulation. It gives you a way to use the value in a variable, and also increment or decrement that value before or after it is used.

But by far the most common place you'll see this is in a `for` loop:

```
for (i = 0; i < 10; i++)
    printf("i is %d\n", i);
```

But more on that later.

3.2.4 The Comma Operator

This is an uncommonly-used way to separate expressions that will run left to right:

```
x = 10, y = 20; // First assign 10 to x, then 20 to y
```

Seems a bit silly, since you could just replace the comma with a semicolon, right?

```
x = 10; y = 20; // First assign 10 to x, then 20 to y
```

But that's a little different. The latter is two separate expressions, while the former is a single expression!

With the comma operator, the value of the comma expression is the value of the rightmost expression:

```
x = (1, 2, 3);

printf("x is %d\n", x); // Prints 3, because 3 is rightmost in the comma list
```

But even that's pretty contrived. One common place the comma operator is used is in `for` loops to do multiple things in each section of the statement:

```
for (i = 0, j = 10; i < 100; i++, j++)
    printf("%d, %d\n", i, j);
```

We'll revisit that later.

3.2.5 Conditional Operators

For Boolean values, we have a raft of standard operators:

```
a == b; // True if a is equivalent to b
a != b; // True if a is not equivalent to b
a < b;  // True if a is less than b
a > b;  // True if a is greater than b
a <= b; // True if a is less than or equal to b
a >= b; // True if a is greater than or equal to b
```

Don't mix up assignment (`=`) with comparison (`==`)! Use two equals to compare, one to assign.

We can use the comparison expressions with `if` statements:

```
if (a <= 10)
    printf("Success!\n");
```

3.2.6 Boolean Operators

We can chain together or alter conditional expressions with Boolean operators for *and*, *or*, and *not*.

Operator	Boolean meaning
<code>&&</code>	and
<code> </code>	or
<code>!</code>	not

An example of Boolean “and”:

```
// Do something if x less than 10 and y greater than 20:
if (x < 10 && y > 20)
    printf("Doing something!\n");
```

An example of Boolean “not”:

```
if (!(x < 12))
    printf("x is not less than 12\n");
```

`!` has higher precedence than the other Boolean operators, so we have to use parentheses in that case.

Of course, that's just the same as:

```
if (x >= 12)
    printf("x is not less than 12\n");
```

but I needed the example!

3.2.7 The `sizeof` Operator

This operator tells you the size (in bytes) that a particular variable or data type uses in memory.

More particularly, it tells you the size (in bytes) that the *type of a particular expression* (which might be just a single variable) uses in memory.

This can be different on different systems, except for `char` and its variants (which are always 1 byte).

And this might not seem very useful now, but we'll be making references to it here and there, so it's worth covering.

Since this computes the number of bytes needed to store a type, you might think it would return an `int`. Or... since the size can't be negative, maybe an `unsigned`?

But it turns out C has a special type to represent the return value from `sizeof`. It's `size_t`, pronounced "size tee"⁷. All we know is that it's an unsigned integer type that can hold the size in bytes of anything you can give to `sizeof`.

`size_t` shows up a lot of different places where counts of things are passed or returned. Think of it as a value that represents a count.

You can take the `sizeof` a variable or expression:

```
int a = 999;

// %zu is the format specifier for type size_t
// If your compiler balks at the "z" part, leave it off

printf("%zu\n", sizeof a);      // Prints 4 on my system
printf("%zu\n", sizeof(2 + 7)); // Prints 4 on my system
printf("%zu\n", sizeof 3.14);  // Prints 8 on my system

// If you need to print out negative size_t values, use %zd
```

Remember: it's the size in bytes of the *type* of the expression, not the size of the expression itself. That's why the size of `2+7` is the same as the size of `a`—they're both type `int`. We'll revisit this number 4 in the very next block of code...

...Where we'll see you can take the `sizeof` a type (note the parentheses are required around a type name, unlike an expression):

```
printf("%zu\n", sizeof(int)); // Prints 4 on my system
printf("%zu\n", sizeof(char)); // Prints 1 on all systems
```

It's important to note that `sizeof` is a *compile-time* operation⁸. The result of the expression is determined entirely at compile-time, not at runtime.

We'll make use of this later on.

3.3 Flow Control

Booleans are all good, but of course we're nowhere if we can't control program flow. Let's take a look at a number of constructs: `if`, `for`, `while`, and `do-while`.

First, a general forward-looking note about statements and blocks of statements brought to you by your local friendly C developer:

After something like an `if` or `while` statement, you can either put a single statement to be executed, or a block of statements to all be executed in sequence.

Let's start with a single statement:

```
if (x == 10) printf("x is 10\n");
```

This is also sometimes written on a separate line. (Whitespace is largely irrelevant in C—it's not like Python.)

⁷The `_t` is short for `type`.

⁸Except for with variable length arrays—but that's a story for another time.

```
if (x == 10)
    printf("x is 10\n");
```

But what if you want multiple things to happen due to the conditional? You can use squirrelly braces to mark a *block* or *compound statement*.

```
if (x == 10) {
    printf("x is 10\n");
    printf("And also this happens when x is 10\n");
}
```

It's a really common style to *always* use squirrelly braces even if they aren't necessary:

```
if (x == 10) {
    printf("x is 10\n");
}
```

Some devs feel the code is easier to read and avoids errors like this where things visually look like they're in the `if` block, but actually they aren't.

```
// BAD ERROR EXAMPLE

if (x == 10)
    printf("This happens if x is 10\n");
    printf("This happens ALWAYS\n"); // Surprise!! Unconditional!
```

`while` and `for` and the other looping constructs work the same way as the examples above. If you want to do multiple things in a loop or after an `if`, wrap them up in squirrelly braces.

In other words, the `if` is going to run the one thing after the `if`. And that one thing can be a single statement or a block of statements.

3.3.1 The `if-else` statement

We've already been using `if` for multiple examples, since it's likely you've seen it in a language before, but here's another:

```
int i = 10;

if (i > 10) {
    printf("Yes, i is greater than 10.\n");
    printf("And this will also print if i is greater than 10.\n");
}

if (i <= 10) printf("i is less than or equal to 10.\n");
```

In the example code, the message will print if `i` is greater than 10, otherwise execution continues to the next line. Notice the squirley braces after the `if` statement; if the condition is true, either the first statement or expression right after the `if` will be executed, or else the collection of code in the squirley braces after the `if` will be executed. This sort of *code block* behavior is common to all statements.

Of course, because C is fun this way, you can also do something if the condition is false with an `else` clause on your `if`:

```
int i = 99;

if (i == 10)
```

```
    printf("i is 10!\n");
else {
    printf("i is decidedly not 10.\n");
    printf("Which irritates me a little, frankly.\n");
}
```

And you can even cascade these to test a variety of conditions, like this:

```
int i = 99;

if (i == 10)
    printf("i is 10!\n");

else if (i == 20)
    printf("i is 20!\n");

else if (i == 99) {
    printf("i is 99! My favorite\n");
    printf("I can't tell you how happy I am.\n");
    printf("Really.\n");
}

else
    printf("i is some crazy number I've never heard of.\n");
```

Though if you're going that route, be sure to check out the `switch` statement for a potentially better solution. The catch is `switch` only works with equality comparisons with constant numbers. The above `if - else` cascade could check inequality, ranges, variables, or anything else you can craft in a conditional expression.

3.3.2 The `while` statement

`while` is your average run-of-the-mill looping construct. Do a thing while a condition expression is true.

Let's do one!

```
// Print the following output:
//
//   i is now 0!
//   i is now 1!
//   [ more of the same between 2 and 7 ]
//   i is now 8!
//   i is now 9!

int i = 0;

while (i < 10) {
    printf("i is now %d!\n", i);
    i++;
}

printf("All done!\n");
```

That gets you a basic loop. C also has a `for` loop which would have been cleaner for that example.

A not-uncommon use of `while` is for infinite loops where you repeat while true:

```
while (1) {
    printf("1 is always true, so this repeats forever.\n");
}
```

3.3.3 The `do-while` statement

So now that we've gotten the `while` statement under control, let's take a look at its closely related cousin, `do-while`.

They are basically the same, except if the loop condition is false on the first pass, `do-while` will execute once, but `while` won't execute at all. In other words, the test to see whether or not to execute the block happens at the *end* of the block with `do-while`. It happens at the *beginning* of the block with `while`.

Let's see by example:

```
// Using a while statement:

i = 10;

// this is not executed because i is not less than 10:
while(i < 10) {
    printf("while: i is %d\n", i);
    i++;
}

// Using a do-while statement:

i = 10;

// this is executed once, because the loop condition is not checked until
// after the body of the loop runs:

do {
    printf("do-while: i is %d\n", i);
    i++;
} while (i < 10);

printf("All done!\n");
```

Notice that in both cases, the loop condition is false right away. So in the `while`, the loop fails, and the following block of code is never executed. With the `do-while`, however, the condition is checked *after* the block of code executes, so it always executes at least once. In this case, it prints the message, increments `i`, then fails the condition, and continues to the "All done!" output.

The moral of the story is this: if you want the loop to execute at least once, no matter what the loop condition, use `do-while`.

All these examples might have been better done with a `for` loop. Let's do something less deterministic—repeat until a certain random number comes up!

```
1 #include <stdio.h> // For printf
2 #include <stdlib.h> // For rand
3
4 int main(void)
5 {
6     int r;
7
```

```

8   do {
9       r = rand() % 100; // Get a random number between 0 and 99
10      printf("%d\n", r);
11  } while (r != 37);    // Repeat until 37 comes up
12  }

```

Side note: did you run that more than once? If you did, did you notice the same sequence of numbers came up again. And again. And again? This is because `rand()` is a pseudorandom number generator that must be *seeded* with a different number in order to generate a different sequence. Look up the `srand()`⁹ function for more details.

3.3.4 The `for` statement

Welcome to one of the most popular loops in the world! The `for` loop!

This is a great loop if you know the number of times you want to loop in advance.

You could do the same thing using just a `while` loop, but the `for` loop can help keep the code cleaner.

Here are two pieces of equivalent code—note how the `for` loop is just a more compact representation:

```

// Print numbers between 0 and 9, inclusive...

// Using a while statement:

i = 0;
while (i < 10) {
    printf("i is %d\n", i);
    i++;
}

// Do the exact same thing with a for-loop:

for (i = 0; i < 10; i++) {
    printf("i is %d\n", i);
}

```

That's right, folks—they do exactly the same thing. But you can see how the `for` statement is a little more compact and easy on the eyes. (JavaScript users will fully appreciate its C origins at this point.)

It's split into three parts, separated by semicolons. The first is the initialization, the second is the loop condition, and the third is what should happen at the end of the block if the loop condition is true. All three of these parts are optional.

```
for (initialize things; loop if this is true; do this after each loop)
```

Note that the loop will not execute even a single time if the loop condition starts off false.

`for`-loop fun fact!

You can use the comma operator to do multiple things in each clause of the `for` loop!

```

for (i = 0, j = 999; i < 10; i++, j--) {
    printf("%d, %d\n", i, j);
}

```

An empty `for` will run forever:

⁹<https://beej.us/guide/bgclr/html/split/stdlib.html#man-srand>


```

for(;;) { // "forever"
    printf("I will print this again and again and again\n" );
    printf("for all eternity until the heat-death of the universe.\n");

    printf("Or until you hit CTRL-C.\n");
}

```

3.3.5 The `switch` Statement

Depending on what languages you're coming from, you might or might not be familiar with `switch`, or C's version might even be more restrictive than you're used to. This is a statement that allows you to take a variety of actions depending on the value of an integer expression.

Basically, it evaluates an expression to an integer value, jumps to the `case` that corresponds to that value. Execution resumes from that point. If a `break` statement is encountered, then execution jumps out of the `switch`.

Here's an example where, for a given number of goats, we print out a gut-feel of how many goats that is.

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int goat_count = 2;
6
7      switch (goat_count) {
8          case 0:
9              printf("You have no goats.\n");
10             break;
11
12         case 1:
13             printf("You have a singular goat.\n");
14             break;
15
16         case 2:
17             printf("You have a brace of goats.\n");
18             break;
19
20         default:
21             printf("You have a bona fide plethora of goats!\n");
22             break;
23     }
24 }

```

In that example, the `switch` will jump to the `case 2` and execute from there. When (if) it hits a `break`, it jumps out of the `switch`.

Also, you might see that `default` label there at the bottom. This is what happens when no cases match.

Every `case`, including `default`, is optional. And they can occur in any order, but it's really typical for `default`, if any, to be listed last.

So the whole thing acts like an `if - else` cascade:

```

if (goat_count == 0)
    printf("You have no goats.\n");
else if (goat_count == 1)

```

```

    printf("You have a singular goat.\n");
else if (goat_count == 2)
    printf("You have a brace of goats.\n");
else
    printf("You have a bona fide plethora of goats!\n");

```

With some key differences:

- `switch` is often faster to jump to the correct code (though the spec makes no such guarantee).
- `if - else` can do things like relational conditionals like `<` and `>=` and floating point and other types, while `switch` cannot.

There's one more neat thing about `switch` that you sometimes see that is quite interesting: *fall through*.

Remember how `break` causes us to jump out of the `switch`?

Well, what happens if we *don't* `break`?

Turns out we just keep on going into the next `case`! Demo!

```

switch (x) {
    case 1:
        printf("1\n");
        // Fall through!
    case 2:
        printf("2\n");
        break;
    case 3:
        printf("3\n");
        break;
}

```

If `x == 1`, this `switch` will first hit `case 1`, it'll print the `1`, but then it just continues on to the next line of code... which prints `2`!

And then, at last, we hit a `break` so we jump out of the `switch`.

if `x == 2`, then we just hit the `case 2`, print `2`, and `break` as normal.

Not having a `break` is called *fall through*.

ProTip: *ALWAYS* put a comment in the code where you intend to fall through, like I did above. It will save other programmers from wondering if you meant to do that.

In fact, this is one of the common places to introduce bugs in C programs: forgetting to put a `break` in your `case`. You gotta do it if you don't want to just roll into the next case¹⁰.

Earlier I said that `switch` works with integer types—keep it that way. Don't use floating point or string types in there. One loophole-ish thing here is that you can use character types because those are secretly integers themselves. So this is perfectly acceptable:

```

char c = 'b';

switch (c) {
    case 'a':
        printf("It's 'a'!\n");
        break;

    case 'b':

```

¹⁰This was considered such a hazard that the designers of the Go Programming Language made `break` the default; you have to explicitly use Go's `fallthrough` statement if you want to fall into the next case.

```
    printf("It's 'b'!\n");  
    break;  
  
    case 'c':  
        printf("It's 'c'!\n");  
        break;  
}
```

Finally, you can use `enum` s in `switch` since they are also integer types. But more on that in the `enum` chapter.

Chapter 4

Functions

“Sir, not in an environment such as this. That’s why I’ve also been programmed for over thirty secondary functions that—”

—C3PO, before being rudely interrupted, reporting a now-unimpressive number of additional functions, *Star Wars* script

Very much like other languages you’re used to, C has the concept of *functions*.

Functions can accept a variety of *arguments* and return a value. One important thing, though: the arguments and return value types are predeclared—because that’s how C likes it!

Let’s take a look at a function. This is a function that takes an `int` as an argument, and returns an `int`.

```
1  #include <stdio.h>
2
3  int plus_one(int n) // The "definition"
4  {
5      return n + 1;
6  }
7
```

The `int` before the `plus_one` indicates the return type.

The `int n` indicates that this function takes one `int` argument, stored in *parameter* `n`. A parameter is a special type of local variable into which the arguments are copied.

I’m going to drive home the point that the arguments are copied into the parameters, here. Lots of things in C are easier to understand if you know that the parameter is a *copy* of the argument, not the argument itself. More on that in a minute.

Continuing the program down into `main()`, we can see the call to the function, where we assign the return value into local variable `j`:

```
8  int main(void)
9  {
10     int i = 10, j;
11
12     j = plus_one(i); // The "call"
13
14     printf("i + 1 is %d\n", j);
15 }
```

Before I forget, notice that I defined the function before I used it. If I hadn't done that, the compiler wouldn't know about it yet when it compiles `main()` and it would have given an unknown function call error. There is a more proper way to do the above code with *function prototypes*, but we'll talk about that later.

Also notice that `main()` is a function!

It returns an `int`.

But what's this `void` thing? This is a keyword that's used to indicate that the function accepts no arguments.

You can also return `void` to indicate that you don't return a value:

```
1  #include <stdio.h>
2
3  // This function takes no arguments and returns no value:
4
5  void hello(void)
6  {
7      printf("Hello, world!\n");
8  }
9
10 int main(void)
11 {
12     hello(); // Prints "Hello, world!"
13 }
```

4.1 Passing by Value

I'd mentioned earlier that when you pass an argument to a function, a copy of that argument gets made and stored in the corresponding parameter.

If the argument is a variable, a copy of the value of that variable gets made and stored in the parameter.

More generally, the entire argument expression is evaluated and its value determined. That value is copied to the parameter.

In any case, the value in the parameter is its own thing. It is independent of whatever values or variables you used as arguments when you made the function call.

So let's look at an example here. Study it and see if you can determine the output before running it:

```
1  #include <stdio.h>
2
3  void increment(int a)
4  {
5      a++;
6  }
7
8  int main(void)
9  {
10     int i = 10;
11
12     increment(i);
13
14     printf("i == %d\n", i); // What does this print?
15 }
```

At first glance, it looks like `i` is `10`, and we pass it to the function `increment()`. There the value gets incremented, so when we print it, it must be `11`, right?

“Get used to disappointment.”

—Dread Pirate Roberts, *The Princess Bride*

But it's not `11`—it prints `10`! How?

It's all about the fact that the expressions you pass to functions get *copied* onto their corresponding parameters. The parameter is a copy, not the original.

So `i` is `10` out in `main()`. And we pass it to `increment()`. The corresponding parameter is called `a` in that function.

And the copy happens, as if by assignment. Loosely, `a = i`. So at that point, `a` is `10`. And out in `main()`, `i` is also `10`.

Then we increment `a` to `11`. But we're not touching `i` at all! It remains `10`.

Finally, the function is complete. All its local variables are discarded (bye, `a`!) and we return to `main()`, where `i` is still `10`.

And we print it, getting `10`, and we're done.

This is why in the previous example with the `plus_one()` function, we `return`ed the locally modified value so that we could see it again in `main()`.

Seems a little bit restrictive, huh? Like you can only get one piece of data back from a function, is what you're thinking. There is, however, another way to get data back; C folks call it *passing by reference* and that's a story we'll tell another time.

But no fancy-schmancy name will distract you from the fact that *EVERYTHING* you pass to a function *WITHOUT EXCEPTION* is copied into its corresponding parameter, and the function operates on that local copy, *NO MATTER WHAT*. Remember that, even when we're talking about this so-called passing by reference.

4.2 Function Prototypes

So if you recall back in the ice age a few sections ago, I mentioned that you had to define the function before you used it, otherwise the compiler wouldn't know about it ahead of time, and would bomb out with an error.

This isn't quite strictly true. You can notify the compiler in advance that you'll be using a function of a certain type that has a certain parameter list. That way the function can be defined anywhere (even in a different file), as long as the *function prototype* has been declared before you call that function.

Fortunately, the function prototype is really quite easy. It's merely a copy of the first line of the function definition with a semicolon tacked on the end for good measure. For example, this code calls a function that is defined later, because a prototype has been declared first:

```

1  #include <stdio.h>
2
3  int foo(void); // This is the prototype!
4
5  int main(void)
6  {
7      int i;
8
9      // We can call foo() here before it's definition because the
10     // prototype has already been declared, above!
11

```

```

12     i = foo();
13
14     printf("%d\n", i); // 3490
15 }
16
17 int foo(void) // This is the definition, just like the prototype!
18 {
19     return 3490;
20 }

```

If you don't declare your function before you use it (either with a prototype or its definition), you're performing something called an *implicit declaration*. This was allowed in the first C standard (C89), and that standard has rules about it, but is no longer allowed today. And there is no legitimate reason to rely on it in new code.

You might notice something about the sample code we've been using... That is, we've been using the good old `printf()` function without defining it or declaring a prototype! How do we get away with this lawlessness? We don't, actually. There is a prototype; it's in that header file `stdio.h` that we included with `#include`, remember? So we're still legit, officer!

4.3 Empty Parameter Lists

You might see these from time to time in older code, but you shouldn't ever code one up in new code. Always use `void` to indicate that a function takes no parameters. There's never¹ a reason to skip this in modern code.

If you're good at just remembering to put `void` in for empty parameter lists in functions and prototypes, you can skip the rest of this section.

There are two contexts for this:

- Omitting all parameters where the function is defined
- Omitting all parameters in a prototype

Let's look at a potential function definition first:

```

void foo() // Should really have a `void` in there
{
    printf("Hello, world!\n");
}

```

While the spec spells out that the behavior in this instance is *as-if* you'd indicated `void` (C11 §6.7.6.3¶14), the `void` type is there for a reason. Use it.

But in the case of a function prototype, there is a *significant* difference between using `void` and not:

```

void foo();
void foo(void); // Not the same!

```

Leaving `void` out of the prototype indicates to the compiler that there is no additional information about the parameters to the function. It effectively turns off all that type checking.

With a prototype **definitely** use `void` when you have an empty parameter list.

¹Never say "never".

Chapter 5

Pointers—Cower In Fear!

“How do you get to Carnegie Hall?”

“Practice!”

—20th-century joke of unknown origin

Pointers are one of the most feared things in the C language. In fact, they are the one thing that makes this language challenging at all. But why?

Because they, quite honestly, can cause electric shocks to come up through the keyboard and physically *weld* your arms permanently in place, cursing you to a life at the keyboard in this language from the 70s!

Really? Well, not really. I’m just trying to set you up for success.

Depending on what language you came from, you might already understand the concept of *references*, where a variable refers to an object of some type.

This is very much the same, except we have to be more explicit with C about when we’re talking about the reference or the thing it refers to.

5.1 Memory and Variables

Computer memory holds data of all kinds, right? It’ll hold `float` s, `int` s, or whatever you have. To make memory easy to cope with, each byte of memory is identified by an integer. These integers increase sequentially as you move up through memory¹. You can think of it as a bunch of numbered boxes, where each box holds a byte² of data. Or like a big array where each element holds a byte, if you come from a language with arrays. The number that represents each box is called its *address*.

Now, not all data types use just a byte. For instance, an `int` is often four bytes, as is a `float`, but it really depends on the system. You can use the `sizeof` operator to determine how many bytes of memory a certain type uses.

```
// %zu is the format specifier for type size_t

printf("an int uses %zu bytes of memory\n", sizeof(int));

// That prints "4" for me, but can vary by system.
```

Memory Fun Facts: When you have a data type (like your typical `int`) that uses more than a byte of memory, the bytes that make up the data are always adjacent to one another in memory. Sometimes they’re in the order that you expect, and sometimes they’re not^a. While C doesn’t

¹Typically. I’m sure there are exceptions out there in the dark corridors of computing history.

²A byte is a number made up of no more than 8 binary digits, or *bits* for short. This means in decimal digits just like grandma used to use, it can hold an unsigned number between 0 and 255, inclusive.

guarantee any particular memory order (it's platform-dependent), it's still generally possible to write code in a way that's platform-independent where you don't have to even consider these pesky byte orderings.

^aThe order that bytes come in is referred to as the *endianness* of the number. The usual suspects are *big-endian* (with the most significant byte first) and *little-endian* (with the most-significant byte last), or, uncommonly now, *mixed-endian* (with the most-significant bytes somewhere else).

So *anyway*, if we can get on with it and get a drum roll and some foreboding music playing for the definition of a pointer, *a pointer is a variable that holds an address*. Imagine the classical score from 2001: A Space Odyssey at this point. Ba bum ba bum ba bum BAAAAH!

Ok, so maybe a bit overwrought here, yes? There's not a lot of mystery about pointers. They are the address of data. Just like an `int` variable can hold the value `12`, a pointer variable can hold the address of data.

This means that all these things mean the same thing, i.e. a number that represents a point in memory:

- Index into memory (if you're thinking of memory like a big array)
- Address
- Location

I'm going to use these interchangeably. And yes, I just threw *location* in there because you can never have enough words that mean the same thing.

And a pointer variable holds that address number. Just like a `float` variable might hold `3.14159`.

Imagine you have a bunch of Post-it® notes all numbered in sequence with their address. (The first one is at index numbered `0`, the next at index `1`, and so on.)

In addition to the number representing their positions, you can also write another number of your choice on each. It could be the number of dogs you have. Or the number of moons around Mars...

...Or, *it could be the index of another Post-it note!*

If you have written the number of dogs you have, that's just a regular variable. But if you wrote the index of another Post-it in there, *that's a pointer*. It points to the other note!

Another analogy might be with house addresses. You can have a house with certain qualities, yard, metal roof, solar, etc. Or you could have the address of that house. The address isn't the same as the house itself. One's a full-blown house, and the other is just a few lines of text. But the address of the house is a *pointer* to that house. It's not the house itself, but it tells you where to find it.

And we can do the same thing in the computer with data. You can have a data variable that's holding some value. And that value is in memory at some address. And you could have a different *pointer variable* hold the address of that data variable.

It's not the data variable itself, but, like with a house address, it tells us where to find it.

When we have that, we say we have a "pointer to" that data. And we can follow the pointer to access the data itself.

(Though it doesn't seem particularly useful yet, this all becomes indispensable when used with function calls. Bear with me until we get there.)

So if we have an `int`, say, and we want a pointer to it, what we want is some way to get the address of that `int`, right? After all, the pointer just holds the *address of* the data. What operator do you suppose we'd use to find the *address of* the `int`?

Well, by a shocking surprise that must come as something of a shock to you, gentle reader, we use the *address-of* operator (which happens to be an ampersand: "&") to find the address of the data. Ampersand.

So for a quick example, we'll introduce a new *format specifier* for `printf()` so you can print a pointer. You know already how `%d` prints a decimal integer, yes? Well, `%p` prints a pointer. Now, this pointer is going to look like a garbage number (and it might be printed in hexadecimal³ instead of decimal), but

³That is, base 16 with digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

it is merely the index into memory the data is stored in. (Or the index into memory that the first byte of data is stored in, if the data is multi-byte.) In virtually all circumstances, including this one, the actual value of the number printed is unimportant to you, and I show it here only for demonstration of the `address-of` operator.

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i = 10;
6
7      printf("The value of i is %d\n", i);
8      printf("And its address is %p\n", (void *)&i);
9  }
```

The above code contains a **cast** where we coerce the type of the expression `&i` to be type `void*`. This is to keep the compiler from throwing a warning here. This is all stuff we haven't covered yet, so just ignore the `(void*)` in the code above for now and pretend it's not there.

On my computer, this prints:

```

The value of i is 10
And its address is 0x7ffddf7072a4
```

If you're curious, that hexadecimal number is 140,727,326,896,068 in decimal (base 10 just like Grandma used to use). That's the index into memory where the variable `i`'s data is stored. It's the address of `i`. It's the location of `i`. It's a pointer to `i`.

Wait—you have 140 terabytes of RAM? Yes! Don't you? But I do fib my buns off; of course I don't (ca. 2024). Modern computers use a miraculous technology called virtual memory^a that makes processes think they have the entire memory space of your computer to themselves, regardless of how much physical RAM backs it up. So even though the address was that huge number, it's being mapped to some lower physical memory address by the virtual memory system of my CPU. This particular computer has 16 GB RAM (again, ca. 2024, but I'm running Linux, so that's plenty). Terabytes of RAM? I'm a teacher, not a dot-com bazillionaire. None of this is anything any of us have to worry about except the part about me not being phenomenally wealthy.

^ahttps://en.wikipedia.org/wiki/Virtual_memory

It's a pointer because it lets you know where `i` is in memory. Like a home address written on a scrap of paper tells you where you can find a particular house, this number indicates to us where in memory we can find the value of `i`. It points to `i`.

Again, we don't really care what the address's exact number is, generally. We just care that it's a pointer to `i`.

5.2 Pointer Types

So... this is all well and good. You can now successfully take the address of a variable and print it on the screen. There's a little something for the ol' resume, right? Here's where you grab me by the scruff of the neck and ask politely what the frick pointers are good for.

Excellent question, and we'll get to that right after these messages from our sponsor.

```

ACME ROBOTIC HOUSING UNIT CLEANING SERVICES. YOUR HOMESTEAD WILL BE
DRAMATICALLY IMPROVED OR YOU WILL BE TERMINATED. MESSAGE ENDS.
```

Welcome back to another installment of Beej's Guide. When we met last we were talking about how to make use of pointers. Well, what we're going to do is store a pointer off in a variable so that we can use

it later. You can identify the *pointer type* because there's an asterisk (*) before the variable name and after its type:

```
1 int main(void)
2 {
3     int i; // i's type is "int"
4     int *p; // p's type is "pointer to an int", or "int-pointer"
5 }
```

Hey, so we have here a variable that is a pointer type, and it can point to other `int` s. That is, it can hold the address of other `int` s. We know it points to `int` s, since it's of type `int*` (read “int-pointer”).

When you do an assignment into a pointer variable, the type of the right hand side of the assignment has to be the same type as the pointer variable. Fortunately for us, when you take the *address-of* a variable, the resultant type is a pointer to that variable type, so assignments like the following are perfect:

```
int i;
int *p; // p is a pointer, but is uninitialized and points to garbage

p = &i; // p is assigned the address of i--p now "points to" i
```

On the left of the assignment, we have a variable of type pointer-to-`int` (`int*`), and on the right side, we have expression of type pointer-to-`int` since `i` is an `int` (because address-of `int` gives you a pointer to `int`). The address of a thing can be stored in a pointer to that thing.

Get it? I know it still doesn't quite make much sense since you haven't seen an actual use for the pointer variable, but we're taking small steps here so that no one gets lost. So now, let's introduce you to the anti-address-of operator. It's kind of like what *address-of* would be like in Bizarro World.

5.3 Dereferencing

A pointer variable can be thought of as *referring* to another variable by pointing to it. It's rare you'll hear anyone in C land talking about “referring” or “references”, but I bring it up just so that the name of this operator will make a little more sense.

When you have a pointer to a variable (roughly “a reference to a variable”), you can use the original variable through the pointer by *dereferencing* the pointer. (You can think of this as “de-pointering” the pointer, but no one ever says “de-pointering”.)

Back to our analogy, this is vaguely like looking at a home address and then going to that house.

Now, what do I mean by “get access to the original variable”? Well, if you have a variable called `i`, and you have a pointer to `i` called `p`, you can use the dereferenced pointer `p` *exactly as if it were the original variable i*!

You almost have enough knowledge to handle an example. The last tidbit you need to know is actually this: what is the dereference operator? It's actually called the *indirection operator*, because you're accessing values indirectly via the pointer. And it is the asterisk, again: `*`. Now, don't get this confused with the asterisk you used in the pointer declaration, earlier. They are the same character, but they have different meanings in different contexts⁴.

Here's a full-blown example:

```
1 #include <stdio.h>
2
3 int main(void)
```

⁴That's not all! It's used in `/*comments*/` and multiplication and in function prototypes with variable length arrays! It's all the same `*`, but the context gives it different meaning.

```

4  {
5      int i;
6      int *p; // this is NOT a dereference--this is a type "int*"
7
8      p = &i; // p now points to i, p holds address of i
9
10     i = 10; // i is now 10
11     *p = 20; // the thing p points to (namely i!) is now 20!!
12
13     printf("i is %d\n", i); // prints "20"
14     printf("i is %d\n", *p); // "20"! dereference-p is the same as i!
15 }

```

Remember that `p` holds the address of `i`, as you can see where we did the assignment to `p` on line 8. What the indirection operator does is tells the computer to *use the object the pointer points to* instead of using the pointer itself. In this way, we have turned `*p` into an alias of sorts for `i`.

Great, but *why*? Why do any of this?

5.4 Passing Pointers as Arguments

Right about now, you're thinking that you have an awful lot of knowledge about pointers, but absolutely zero application, right? I mean, what use is `*p` if you could just simply say `i` instead?

Well, my friend, the real power of pointers comes into play when you start passing them to functions. Why is this a big deal? You might recall from before that you could pass all kinds of arguments to functions and they'd be dutifully copied into parameters, and then you could manipulate local copies of those variables from within the function, and then you could return a single value.

What if you wanted to bring back more than one single piece of data from the function? I mean, you can only return one thing, right? What if I answered that question with another question? ...Er, two questions?

What happens when you pass a pointer as an argument to a function? Does a copy of the pointer get put into its corresponding parameter? *You bet your sweet peas it does.* Remember how earlier I rambled on and on about how *EVERY SINGLE ARGUMENT* gets copied into parameters and the function uses a copy of the argument? Well, the same is true here. The function will get a copy of the pointer.

But, and this is the clever part: we will have set up the pointer in advance to point at a variable... and then the function can dereference its copy of the pointer to get back to the original variable! The function can't see the variable itself, but it can certainly dereference a pointer to that variable!

This is analogous to writing a home address on a piece of paper, and then copying that onto another piece of paper. You now have *two* pointers to that house, and both are equally good at getting you to the house itself.

In the case of a function call, one of the copies is stored in a pointer variable out in the calling scope, and the other is stored in a pointer variable that is the parameter of the function.

Example! Let's revisit our old `increment()` function, but this time let's make it so that it actually increments the value out in the caller.

```

1  #include <stdio.h>
2
3  void increment(int *p) // note that it accepts a pointer to an int
4  {
5      *p = *p + 1;       // add one to the thing p points to
6  }
7
8  int main(void)

```

```

9  {
10     int i = 10;
11     int *j = &i; // note the address-of; turns it into a pointer to i
12
13     printf("i is %d\n", i); // prints "10"
14     printf("i is also %d\n", *j); // prints "10"
15
16     increment(j); // j is an int*--to i
17
18     printf("i is %d\n", i); // prints "11"!
19 }

```

Ok! There are a couple things to see here... not the least of which is that the `increment()` function takes an `int*` as an argument. We pass it an `int*` in the call by changing the `int` variable `i` to an `int*` using the `address-of` operator. (Remember, a pointer holds an address, so we make pointers to variables by running them through the `address-of` operator.)

The `increment()` function gets a copy of the pointer. Both the original pointer `j` (in `main()`) and the copy of that pointer `p` (the parameter in `increment()`) point to the same address, namely the one holding the value `i`. (Again, by analogy, like two pieces of paper with the same home address written on them.) Dereferencing either will allow you to modify the original variable `i`! The function can modify a variable in another scope! Rock on!

The above example is often more concisely written in the call just by using `address-of` right in the argument list:

```

printf("i is %d\n", i); // prints "10"
increment(&i);
printf("i is %d\n", i); // prints "11"!

```

As a general rule, if you want the function to modify the thing that you're passing in such that you see the result, you'll have to pass a pointer to that thing.

5.5 The `NULL` Pointer

Any pointer variable of any pointer type can be set to a special value called `NULL`. This indicates that this pointer doesn't point to anything.

```

int *p;

p = NULL;

```

Since it doesn't point to a value, dereferencing it is undefined behavior, and probably will result in a crash:

```

int *p = NULL;

*p = 12; // CRASH or SOMETHING PROBABLY BAD. BEST AVOIDED.

```

Despite being called the billion dollar mistake by its creator⁵, the `NULL` pointer is a good sentinel value⁶ and general indicator that a pointer hasn't yet been initialized.

(Of course, like other variables, the pointer points to garbage unless you explicitly assign it to point to an address or `NULL`.)

⁵https://en.wikipedia.org/wiki/Null_pointer#History

⁶https://en.wikipedia.org/wiki/Sentinel_value

5.6 A Note on Declaring Pointers

The syntax for declaring a pointer can get a little weird. Let's look at this example:

```
int a;
int b;
```

We can condense that into a single line, right?

```
int a, b; // Same thing
```

So `a` and `b` are both `int`s. No problem.

But what about this?

```
int a;
int *p;
```

Can we make that into one line? We can. But where does the `*` go?

The rule is that the `*` goes in front of any variable that is a pointer type. That is, the `*` is *not* part of the `int` in this example. it's a part of variable `p`.

With that in mind, we can write this:

```
int a, *p; // Same thing
```

It's important to note that the following line does *not* declare two pointers:

```
int *p, q; // p is a pointer to an int; q is just an int.
```

This can be particularly insidious-looking if the programmer writes this following (valid) line of code which is functionally identical to the one above.

```
int* p, q; // p is a pointer to an int; q is just an int.
```

So take a look at this and determine which variables are pointers and which are not:

```
int *a, b, c, *d, e, *f, g, h, *i;
```

I'll drop the answer in a footnote⁷.

5.7 `sizeof` and Pointers

Just a little bit of syntax here that might be confusing and you might see from time to time.

Recall that `sizeof` operates on the *type* of the expression.

```
int *p;

// Prints size of an 'int'
printf("%zu\n", sizeof(int));

// p is type 'int *', so prints size of 'int*'
printf("%zu\n", sizeof p);
```

⁷The pointer type variables are `a`, `d`, `f`, and `i`, because those are the ones with `*` in front of them.

```
// *p is type 'int', so prints size of 'int'  
printf("%zu\n", sizeof *p);
```

You might see code in the wild with that last `sizeof` in there. Just remember that `sizeof` is all about the type of the expression, not the variables in the expression themselves.

Chapter 6

Arrays

“Should array indices start at 0 or 1? My compromise of 0.5 was rejected without, I thought, proper consideration.”

—Stan Kelly-Bootle, computer scientist

Luckily, C has arrays. I mean, I know it’s considered a low-level language¹ but it does at least have the concept of arrays built-in. And since a great many languages drew inspiration from C’s syntax, you’re probably already familiar with using `[` and `]` for declaring and using arrays.

But C only *barely* has arrays! As we’ll find out later, arrays are just syntactic sugar in C—they’re actually all pointers and stuff deep down. *Freak out!* But for now, let’s just use them as arrays. *Phew.*

6.1 Easy Example

Let’s just crank out an example:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i;
6      float f[4]; // Declare an array of 4 floats
7
8      f[0] = 3.14159; // Indexing starts at 0, of course.
9      f[1] = 1.41421;
10     f[2] = 1.61803;
11     f[3] = 2.71828;
12
13     // Print them all out:
14
15     for (i = 0; i < 4; i++) {
16         printf("%f\n", f[i]);
17     }
18 }
```

When you declare an array, you have to give it a size. And the size has to be fixed².

In the above example, we made an array of 4 `float` s. The value in the square brackets in the declaration lets us know that.

¹These days, anyway.

²Again, not really, but variable-length arrays—of which I’m not really a fan—are a story for another time.

Later on in subsequent lines, we access the values in the array, setting them or getting them, again with square brackets.

Hopefully this looks familiar from languages you already know!

6.2 Getting the Length of an Array

You can't...ish. C doesn't record this information³. You have to manage it separately in another variable.

When I say "can't", I actually mean there are some circumstances when you *can*. There is a trick to get the number of elements in an array in the scope in which an array is declared. But, generally speaking, this won't work the way you want if you pass the array to a function⁴.

Let's take a look at this trick. The basic idea is that you take the `sizeof` the array, and then divide that by the size of each element to get the length. For example, if an `int` is 4 bytes, and the array is 32 bytes long, there must be room for $\frac{32}{4}$ or 8 `int` s in there.

```
int x[12]; // 12 ints

printf("%zu\n", sizeof x); // 48 total bytes
printf("%zu\n", sizeof(int)); // 4 bytes per int

printf("%zu\n", sizeof x / sizeof(int)); // 48/4 = 12 ints!
```

If it's an array of `char` s, then `sizeof` the array is the number of elements, since `sizeof(char)` is defined to be 1. For anything else, you have to divide by the size of each element.

But this trick only works in the scope in which the array was defined. If you pass the array to a function, it doesn't work. Even if you make it "big" in the function signature:

```
void foo(int x[12])
{
    printf("%zu\n", sizeof x); // 8?! What happened to 48?
    printf("%zu\n", sizeof(int)); // 4 bytes per int

    printf("%zu\n", sizeof x / sizeof(int)); // 8/4 = 2 ints?? WRONG.
}
```

This is because when you "pass" arrays to functions, you're only passing a pointer to the first element, and that's what `sizeof` measures. More on this in the Passing Single Dimensional Arrays to Functions section, below.

One more thing you can do with `sizeof` and arrays is get the size of an array of a fixed number of elements without declaring the array. This is like how you can get the size of an `int` with `sizeof(int)`.

For example, to see how many bytes would be needed for an array of 48 `double` s, you can do this:

```
sizeof(double [48]);
```

6.3 Array Initializers

You can initialize an array with constants ahead of time:

³Since arrays are just pointers to the first element of the array under the hood, there's no additional information recording the length.

⁴Because when you pass an array to a function, you're actually just passing a pointer to the first element of that array, not the "entire" array.

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i;
6      int a[5] = {22, 37, 3490, 18, 95}; // Initialize with these values
7
8      for (i = 0; i < 5; i++) {
9          printf("%d\n", a[i]);
10     }
11 }

```

You should never have more items in your initializer than there is room for in the array, or the compiler will get cranky:

```

foo.c: In function 'main':
foo.c:6:39: warning: excess elements in array initializer
    6 |     int a[5] = {22, 37, 3490, 18, 95, 999};
      |                                     ^~~
foo.c:6:39: note: (near initialization for 'a')

```

But (fun fact!) you can have *fewer* items in your initializer than there is room for in the array. The remaining elements in the array will be automatically initialized with zero. This is true in general for all types of array initializers: if you have an initializer, anything not explicitly set to a value will be set to zero.

```

int a[5] = {22, 37, 3490};

// is the same as:

int a[5] = {22, 37, 3490, 0, 0};

```

It's a common shortcut to see this in an initializer when you want to set an entire array to zero:

```
int a[100] = {0};
```

Which means, “Make the first element zero, and then automatically make the rest zero, as well.”

You can set specific array elements in the initializer, as well, by specifying an index for the value! When you do this, C will happily keep initializing subsequent values for you until the initializer runs out, filling everything else with `0`.

To do this, put the index in square brackets with an `=` after, and then set the value.

Here's an example where we build an array:

```
int a[10] = {0, 11, 22, [5]=55, 66, 77};
```

Because we listed index 5 as the start for `55`, the resulting data in the array is:

```
0 11 22 0 0 55 66 77 0 0
```

You can put simple constant expressions in there, as well.

```

#define COUNT 5

int a[COUNT] = {[COUNT-3]=3, 2, 1};

```

which gives us:

```
0 0 3 2 1
```

Lastly, you can also have C compute the size of the array from the initializer, just by leaving the size off:

```
int a[3] = {22, 37, 3490};

// is the same as:

int a[] = {22, 37, 3490}; // Left the size off!
```

6.4 Out of Bounds!

C doesn't stop you from accessing arrays out of bounds. It might not even warn you.

Let's steal the example from above and keep printing off the end of the array. It only has 5 elements, but let's try to print 10 and see what happens:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i;
6      int a[5] = {22, 37, 3490, 18, 95};
7
8      for (i = 0; i < 10; i++) { // BAD NEWS: printing too many elements!
9          printf("%d\n", a[i]);
10     }
11 }
```

Running it on my computer prints:

```
22
37
3490
18
95
32765
1847052032
1780534144
-56487472
21890
```

Yikes! What's that? Well, turns out printing off the end of an array results in what C developers call *undefined behavior*. We'll talk more about this beast later, but for now it means, "You've done something bad, and anything could happen during your program run."

And by anything, I mean typically things like finding zeroes, finding garbage numbers, or crashing. But really the C spec says in this circumstance the compiler is allowed to emit code that does *anything*⁵.

Short version: don't do anything that causes undefined behavior. Ever⁶.

⁵In the good old MS-DOS days before memory protection was a thing, I was writing some particularly abusive C code that deliberately engaged in all kinds of undefined behavior. But I knew what I was doing, and things were working pretty well. Until I made a misstep that caused a lockup and, as I found upon reboot, nuked all my BIOS settings. That was fun. (Shout-out to @man for those fun times.)

⁶There are a lot of things that cause undefined behavior, not just out-of-bounds array accesses. This is what makes the C language so *exciting*.

6.5 Multidimensional Arrays

You can add as many dimensions as you want to your arrays.

```
int a[10];
int b[2][7];
int c[4][5][6];
```

These are stored in memory in row-major order⁷. This means with a 2D array, the first index listed indicates the row, and the second the column.

You can also use initializers on multidimensional arrays by nesting them:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int row, col;
6
7      int a[2][5] = {          // Initialize a 2D array
8          {0, 1, 2, 3, 4},
9          {5, 6, 7, 8, 9}
10     };
11
12     for (row = 0; row < 2; row++) {
13         for (col = 0; col < 5; col++) {
14             printf("(%d,%d) = %d\n", row, col, a[row][col]);
15         }
16     }
17 }
```

For output of:

```
(0,0) = 0
(0,1) = 1
(0,2) = 2
(0,3) = 3
(0,4) = 4
(1,0) = 5
(1,1) = 6
(1,2) = 7
(1,3) = 8
(1,4) = 9
```

And you can initialize with explicit indexes:

```
// Make a 3x3 identity matrix

int a[3][3] = {[0][0]=1, [1][1]=1, [2][2]=1};
```

which builds a 2D array like this:

```
1 0 0
0 1 0
0 0 1
```

⁷https://en.wikipedia.org/wiki/Row-_and_column-major_order

6.6 Arrays and Pointers

[Casually] So... I kinda might have mentioned up there that arrays were pointers, deep down? We should take a shallow dive into that now so that things aren't completely confusing. Later on, we'll look at what the real relationship between arrays and pointers is, but for now I just want to look at passing arrays to functions.

6.6.1 Getting a Pointer to an Array

I want to tell you a secret. Generally speaking, when a C programmer talks about a pointer to an array, they're talking about a pointer *to the first element* of the array⁸.

So let's get a pointer to the first element of an array.

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int a[5] = {11, 22, 33, 44, 55};
6      int *p;
7
8      p = &a[0]; // p points to the array
9                  // Well, to the first element, actually
10
11     printf("%d\n", *p); // Prints "11"
12 }
```

This is so common to do in C that the language allows us a shorthand:

```

1  p = &a[0]; // p points to the array
2
3  // is the same as:
4
5  p = a;      // p points to the array, but much nicer-looking!
```

Just referring to the array name in isolation is the same as getting a pointer to the first element of the array! We're going to use this extensively in the upcoming examples.

But hold on a second— isn't `p` an `int*`? And `*p` gives us `11`, same as `a[0]`? Yessss. You're starting to get a glimpse of how arrays and pointers are related in C. (We'll talk about this a lot more in the Pointers II chapter, under Array/Pointer Equivalence.)

6.6.2 Passing Single Dimensional Arrays to Functions

Let's do an example with a single dimensional array. I'm going to write a couple functions that we can pass the array to that do different things.

Prepare for some mind-blowing function signatures!

```

1  #include <stdio.h>
2
3  // Passing as a pointer to the first element
4  void times2(int *a, int len)
5  {
6      for (int i = 0; i < len; i++)
7          printf("%d\n", a[i] * 2);
8  }
```

⁸This is technically incorrect, as a pointer to an array and a pointer to the first element of an array have different types. But we can burn that bridge when we get to it.

```

9
10 // Same thing, but using array notation
11 void times3(int a[], int len)
12 {
13     for (int i = 0; i < len; i++)
14         printf("%d\n", a[i] * 3);
15 }
16
17 // Same thing, but using array notation with size
18 void times4(int a[5], int len)
19 {
20     for (int i = 0; i < len; i++)
21         printf("%d\n", a[i] * 4);
22 }
23
24 int main(void)
25 {
26     int x[5] = {11, 22, 33, 44, 55};
27
28     times2(x, 5);
29     times3(x, 5);
30     times4(x, 5);
31 }

```

All those methods of listing the array as a parameter in the function are identical.

```

void times2(int *a, int len)
void times3(int a[], int len)
void times4(int a[5], int len)

```

In usage by C regulars, the first is the most common, by far.

And, in fact, in the latter situation, the compiler doesn't even care what number you pass in (other than it has to be greater than zero⁹). It doesn't enforce anything at all.

Now that I've said that, the size of the array in the function declaration actually *does* matter when you're passing multidimensional arrays into functions, but let's come back to that.

6.6.3 Changing Arrays in Functions

We've said that arrays are just pointers in disguise. This means that if you pass an array to a function, you're likely passing a pointer to the first element in the array.

But if the function has a pointer to the data, it is able to manipulate that data! So changes that a function makes to an array will be visible back out in the caller.

Here's an example where we pass a pointer to an array to a function, the function manipulates the values in that array, and those changes are visible out in the caller.

```

1 #include <stdio.h>
2
3 void double_array(int *a, int len)
4 {
5     // Multiply each element by 2

```

⁹C11 §6.7.6.2^{¶1} requires it be greater than zero. But you might see code out there with arrays declared of zero length at the end of `struct` s and GCC is particularly lenient about it unless you compile with `-pedantic`. This zero-length array was a hackish mechanism for making variable-length structures. Unfortunately, it's technically undefined behavior to access such an array even though it basically worked everywhere. C99 codified a well-defined replacement for it called *flexible array members*, which we'll chat about later.

```

6      //
7      // This doubles the values in x in main() since x and a both point
8      // to the same array in memory!
9
10     for (int i = 0; i < len; i++)
11         a[i] *= 2;
12 }
13
14 int main(void)
15 {
16     int x[5] = {1, 2, 3, 4, 5};
17
18     double_array(x, 5);
19
20     for (int i = 0; i < 5; i++)
21         printf("%d\n", x[i]); // 2, 4, 6, 8, 10!
22 }

```

Even though we passed the array in as parameter `a` which is type `int*`, look at how we access it using array notation with `a[i]`! Whaaaaat. This is totally allowed.

Later when we talk about the equivalence between arrays and pointers, we'll see how this makes a lot more sense. For now, it's enough to know that functions can make changes to arrays that are visible out in the caller.

6.6.4 Passing Multidimensional Arrays to Functions

The story changes a little when we're talking about multidimensional arrays. C needs to know all the dimensions (except the first one) so it has enough information to know where in memory to look to find a value.

Here's an example where we're explicit with all the dimensions:

```

1  #include <stdio.h>
2
3  void print_2D_array(int a[2][3])
4  {
5      for (int row = 0; row < 2; row++) {
6          for (int col = 0; col < 3; col++)
7              printf("%d ", a[row][col]);
8          printf("\n");
9      }
10 }
11
12 int main(void)
13 {
14     int x[2][3] = {
15         {1, 2, 3},
16         {4, 5, 6}
17     };
18
19     print_2D_array(x);
20 }

```

But in this case, these two¹⁰ are equivalent:

¹⁰This is also equivalent: `void print_2D_array(int (*a)[3])`, but that's more than I want to get into right now.

```
void print_2D_array(int a[2][3])  
void print_2D_array(int a[][3])
```

The compiler really only needs the second dimension so it can figure out how far in memory to skip for each increment of the first dimension. In general, it needs to know all the dimensions except the first one.

Also, remember that the compiler does minimal compile-time bounds checking (if you're lucky), and C does zero runtime checking of bounds. No seat belts! Don't crash by accessing array elements out of bounds!